

- DATA STRUCTURES AND ALGORITHMS – ECCI 3104
 - ANSWERS IN ACRONYM & MNEMONIC FORMAT
 - QUESTION ONE – QUICK REFERENCE GUIDE
 - QUESTION TWO – SORTING SNAPSHOT
 - QUESTION THREE – HEAP & BST ESSENTIALS
 - QUESTION FOUR – SEARCH & ANALYSIS
 - QUESTION FIVE – GRAPHS & APPLICATIONS
 - MNEMONIC SUMMARY CHEAT SHEET

DATA STRUCTURES AND ALGORITHMS – ECCI 3104

ANSWERS IN ACRONYM & MNEMONIC FORMAT

QUESTION ONE – QUICK REFERENCE GUIDE

1a) Algorithm Complexity (Importance: PEC)

- Performance Prediction
- Efficiency Comparison
- Computational Resource Management

1b) PUSH Flowchart (Steps: C-I-P-C)

1. Check if stack is full (Overflow?).
2. Increment the TOP pointer.
3. Place new element at $S[TOP]$.
4. Confirm operation success.

1c) Binary Tree (Distinctions: DH / CB)

- Depth: Distance from Root.

- **Height:** Longest path to a **Leaf**.
- **Complete Tree:** All levels full, last level **Left-packed**.
- **Balanced Tree** (e.g., AVL): **Height difference** ≤ 1 for **Any** node.

1d) Data Structure Selection (Factors: NORT)

1. **Nature** of the data (Linear, Hierarchical, Networked).
2. **Operations** required (Search, Insert, Delete, Traverse).
3. **Runtime** constraints (Time Complexity: Best/Avg/Worst-case).
4. **Throughput & Memory** (Space Efficiency).

1e) BFS vs. DFS (Memory Use & Structure)

- **BFS:** Uses a **Queue**. **Level** order. High memory (**Wide**).
- **DFS:** Uses a **Stack**. **Depth** first. Lower memory (**Deep**).

1f) Find Largest Program (Steps: I-D-F)

1. Input 10 values into array.
2. **Display** the entered values.
3. **Find** largest via linear scan.

QUESTION TWO – SORTING SNAPSHOT

2a) In-Place vs. Out-of-Place (Keywords)

- **In-Place:** **Modifies** original, **Low** extra space (Bubble, Selection, Heap).
- **Out-of-Place:** **Creates** copy, **High** extra space (Merge, Bucket).

2b) Time Complexity Cases (What they show)

- **Worst-Case:** **Upper** bound / **Guarantee**.
- **Average-Case:** **Expected** performance.
- **Best-Case:** **Lower** bound / **Ideal** scenario.

2c) Bubble vs. Selection Sort (Core Action)

- **Bubble Sort:** **Adjacent** **Swap** (repeatedly).
- **Selection Sort:** **Select** **Min** & **Swap** (once per pass).

2d) Bubble Sort Program (Phases: I-S-O)

1. Input array.
 2. Sort using nested loops & swaps.
 3. Output before & after.
-

QUESTION THREE – HEAP & BST ESSENTIALS

3a) Heap Properties (The 3 C's)

1. Complete Binary Tree.
2. Correct Order (Max/Min).
3. Compact Array Representation.

3b) BST vs. MAX Heap (Rules)

- **BST Rule:** Left < Parent < Right.
- **MAX Heap Rule:** Parent $\geq$ Children & Complete tree.

3c) BST Traversals (Order of L, R, N)

- **INorder:** Left, Node, Right $\rightarrow$ Sorted output.
- **PREorder:** Node, Left, Right.
- **POSTorder:** Left, Right, Node.
- **LEVEL** order: By Level (BFS).

3d) Heap Applications (Key Uses: PQ-HS)

1. Priority Queues (Scheduling, Dijkstra's).
 2. HeapSort (In-place $O(n \log n)$ sorting).
-

QUESTION FOUR – SEARCH & ANALYSIS

4a) Big O Purpose (What it does: CUP)

- Compare algorithms.
- Upper bound on growth.

- Predict scalability.

4b) Complexity Cases (Focus)

- **Best-Case** → **L**ower bound.
- **Worst-Case** → **G**uarantee.
- **Average-Case** → **E**xpected.

4d) Linear Search Complexity (Best/Worst/Avg)

- **Best**: $O(1)$ (First element).
- **Worst/Avg**: $O(n)$ (Scan up to n elements).

4e) Linear Search Program (Flow: G-S-R)

1. **G**enerate random array.
 2. **S**earch for key linearly.
 3. **R**eport found/index or not found.
-

QUESTION FIVE – GRAPHS & APPLICATIONS

5a) Social Media Graphs (Model & Propagate)

- **Modeling (VERTICES & EDGES)**:
 - Vertices = Users.
 - Edges = Follows/Friendships.
 - Enables: **R**ecommendations, **I**nfluencer ID, **C**ommunity detection.
- **Propagation (CASCADE)**:
 - Content flows along edges.
 - Analyzed via BFS/DFS.
 - Shares = Cascades.
 - Crucial for ads & **E**pidemiology models.

5b) Graph Representations (Two Ways: M & L)

- **Matrix**: 2D array, **1** = edge.
- **Linked List**: Array of lists, each list = neighbors.

5c) Heap for SJF (Why: EES)

- Efficient $O(\log n)$ inserts.
 - Efficient $O(\log n)$ extract-min.
 - Supports core SJF operation (always get shortest job).
-

MNEMONIC SUMMARY CHEAT SHEET

- **Algorithm Analysis:** Think **PEC** (Predict, Compare, Conserve).
- **Data Structure Choice:** Ask **NORT** (Nature, Ops, Runtime, Throughput).
- **Tree Height vs. Depth:** **Down** from **Root** vs. **Long** to **Leaf**.
- **BFS vs. DFS:** **BFS** = **Queue**, **Wide**. **DFS** = **Stack**, **Deep**.
- **Sorting Types:** **In-Place** = **Modify**. **Out-of-Place** = **Copy**.
- **Heap Properties:** The **3 C's** (Complete, Correct Order, Compact Array).
- **BST Traversal:** **IN** = **LNR** (Sorted). **PRE** = **NLR**. **POST** = **LRN**.
- **Linear Search:** **Best** = 1st, **Worst/Avg** = Scan **All**.
- **Graphs in Social Media:** **Vertices**, **Edges**, **Cascades**.
- **SJF with Heap:** $\log n$ for **Insert** & **Extract**.